

UNIT-I

Introduction to Software Engineering



Topic Covered

- Define s/w Engineering
- Characteristics of s/w
- Evolving Role of Software
- Changing nature of software
- Software Myths
- Generic View of Process
 1. Layered Technology.
 2. Process Framework.
 3. CMMI.

Definition Software Engineering

Software Engineering is the process of Designing, Developing, Testing, and Maintaining S/W Applications. It's a branch of Computer Science that uses Engineering principles and Programming language to build s/w for end users.

OR

Software engineering is defined as analyzing user requirements and then designing, building, and testing software applications that will satisfy those requirements.

Characteristics of s/w

6 Types of s/w characteristics

1. Functionality
2. Usability (User-friendly)
3. Efficiency
4. Reliability
5. Maintainability
6. Portability

- 1. Functionality**
 - 1. suitability**
 - 2. accuracy**
 - 3. interoperability**
 - 4. compliance**
 - 5. security**
- 2. Efficiency**
 - 1. In time**
 - 2. In Resource**

Usability (User-friendly)

1. understandability

2. learnability

3. operability

Reliability

1. Recoverability

2. Fault tolerance

3. Maturity

- Maintainability

1. Testability

2. Stability

3. Changability

4. operability

Portability

1. Adaptability

2. Installability

3. Replaceability

Evolving Role of Software

- Software is a product
- Software is a vehicle for delivering a product
- Definition of Software
- Hardware vs. Software
- Manufacturing vs. Development
- Failure curve for Hardware and Software
- Component Based vs. Custom Built

Evolving Role of Software

1. Software is a product

- Transforms information - produces, manages, modifies, displays, or transmits information
- Delivers computing potential of hardware and networks

2. Software is a vehicle for delivering a product

- Controls other programs (operating system)
- Effects communications (networking software)
- Helps build other software (software tools & environments)

3. What is Software?

Software can define as:

- Instruction – executed provide desire features, function & performance.
- Data structure – to adequately manipulate operation.
- Documents – operation and use of the program.

Software products may be developed for a particular customer or may be developed for a general market.

4. Hardware vs. Software

Hardware

- ☐ Manufactured
- ☐ wear out
- ☐ Built using components
- ☐ Relatively simple

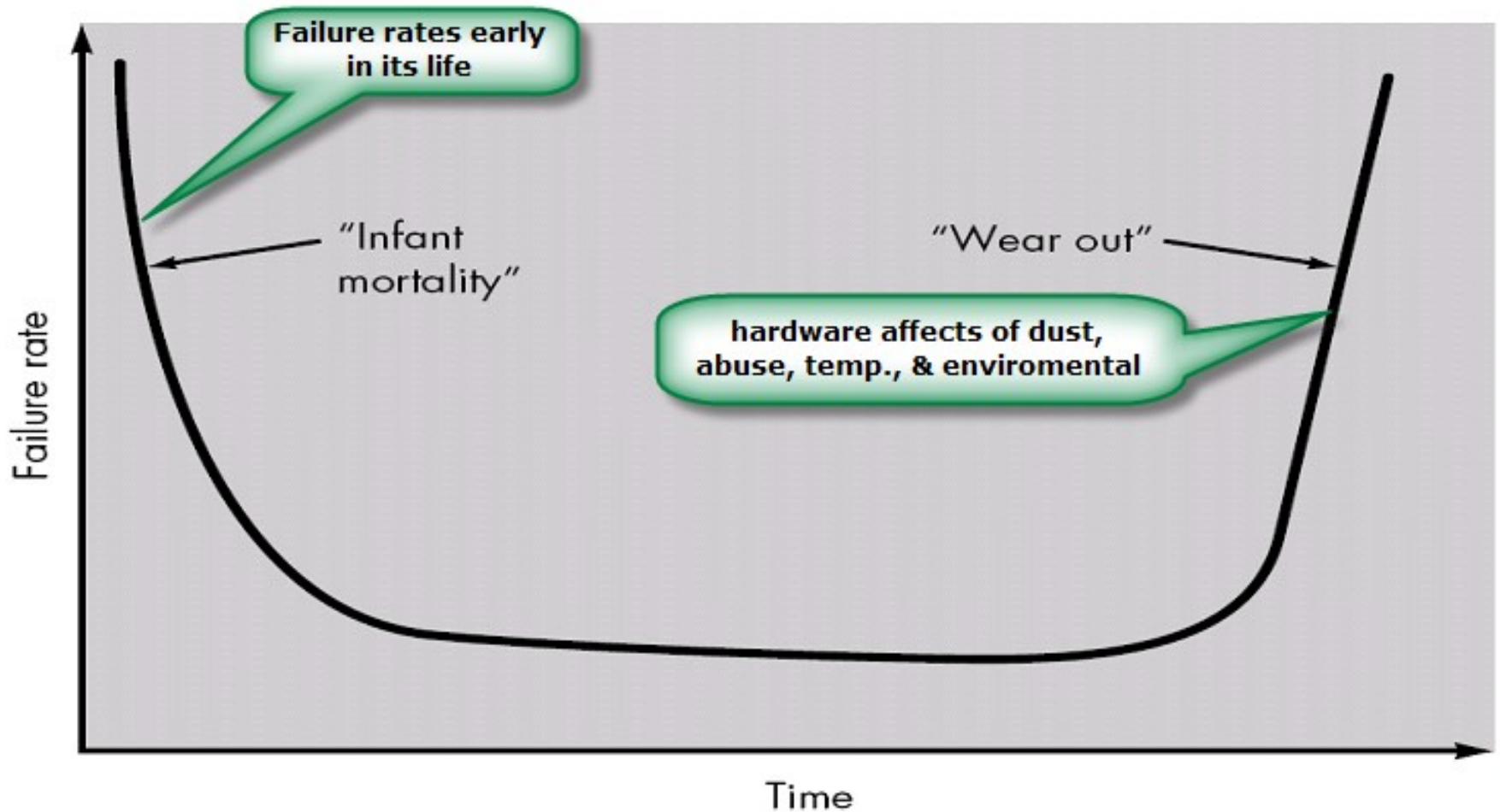
Software

- ☐ Developed/ engineered
- ☐ deteriorate
- ☐ Custom built
- ☐ Complex

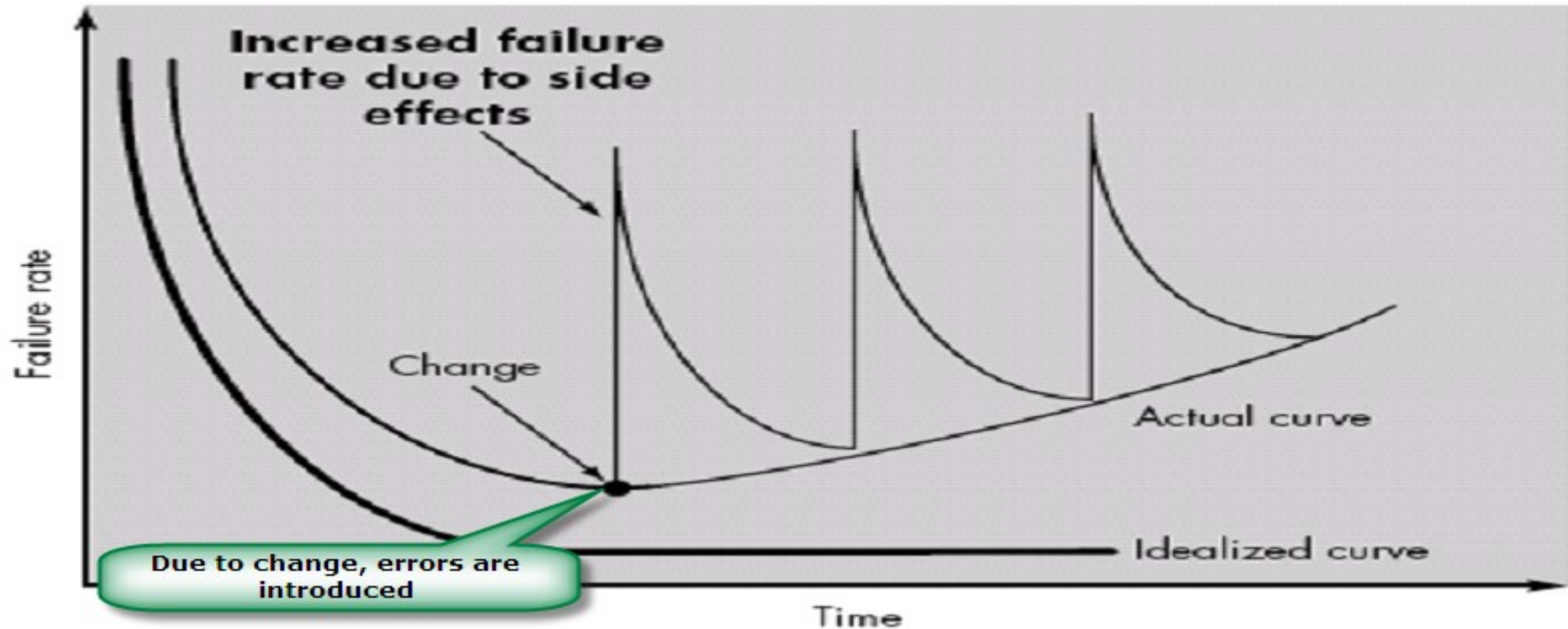
5. Manufacturing vs. Development

- Once a hardware product has been manufactured, it is difficult or impossible to modify. In contrast, software products are routinely modified and upgraded.
- In hardware, hiring more people allows you to accomplish more work, but the same does not necessarily hold true in software engineering.
- Unlike hardware, software costs are concentrated in design rather than production.

6.a. Failure curve for Hardware



6.b. Failure curve for Software



When a hardware component wears out, it is replaced by a spare part. There are no software spare parts. Every software failure indicates an error in design or in the process through which design was translated into machine executable code. Therefore, software maintenance involves considerably more complexity

7. Component Based vs. Custom Built

- Hardware products typically employ many standardized design components.
- Most software continues to be custom built.
- The software industry does seem to be moving (slowly) toward component-based construction.

Changing nature of software

1. System software
2. Application software
3. Engineering/scientific software
4. Embedded software
5. Product line software
6. Web applications
7. Artificial intelligence software

1. System Software:

- System software is a collection of programs written to service other programs.
- It is characterized by heavy interaction with computer hardware; heavy usage by multiple users; concurrent operation that requires scheduling, resource sharing, complex data structures; and multiple external interfaces.

Ex. Compilers, operating system, drivers etc.

2. Application Software :

- Application software consists of standalone programs that solve a specific business need.
- Application software is used to control the business function in real time.

Ex: Microsoft Office, VLC media player, Jet audio, Google chrome

3. Engineering /Scientific software:

- Characterized by "number crunching" algorithms.
- **Scientific and engineering software** satisfies the needs of a **scientific** or **engineering** user to perform enterprise-specific tasks. Such **software** is written for specific applications using principles, techniques, and formulae particular to that field.
- Ex. Computer-Aided Design (CAD), MATLAB, AUTOCAD

4. Embedded Software:

- Controlling devices and machines that are different from traditional computers using software engineering.
- Designing, developing, and testing software embedded within hardware devices or systems.

Ex. keypad control for a microwave oven, Traffic control systems.

5. Product line software:

- Designed to provide a specific capability for use by many different customers, product line software can focus on a limited marketplace.

Ex. Word processing, spreadsheet, CG, multimedia, etc.

6. Web Applications:

- Web apps can be little more than a set of linked hypertext files.
- It is evolving into sophisticated computing environments that provide standalone features and functions and are integrated with corporate database and business applications.

Ex: online forms, shopping carts, word processors, spreadsheets, video and photo editing.

7. Artificial Intelligence software

- AI software makes use of non-numerical algorithms to solve complex problems and straightforward analysis
- **Ex.** Robotics, expert systems, game playing, Manufacturing robots, Smart assistants, healthcare management, Disease mapping, Automated financial investing, Virtual travel booking agent, Social media monitoring

Software Myths

Definition: Beliefs about software and the process used to build it. Myths have number of attributes that have made them insidious (i.e. dangerous).

- Misleading Attitudes - caused serious problem for managers and technical people.

There are three types of S/W Myths

1. Management myths

2. Customer myths

3. Practitioner's/Developer's myths

1. Management myths

Managers in most disciplines, are often under pressure to maintain budgets, keep schedules on time, and improve quality.

Myth1: If we get behind schedule, we can add more programmers and catch up

Reality: Software development is not a mechanistic process like manufacturing. Adding people to a late software project makes it later.

- People can be added but only in a planned and well-coordinated manner

Myth2: If I decide to outsource the software project to a third party, I can just relax and let that firm build it.

Reality: If an organization does not understand how to manage and control software projects internally, it will invariably struggle when it outsource software projects

- **The more features we add, the better the product."**
- **Reality:** Focusing on user needs and core functionalities often leads to a more valuable product than simply adding features.

2. Customer Myths

Customer may be a person from inside or outside the company that has requested software under contract.

- **Myth: Once the software is developed, it's done."**
- **Reality:** Software requires ongoing maintenance, updates, and support, even after deployment.

Myth: Project requirements continually change, but change can be easily accommodated because software is flexible.

Reality: Customer can review requirements and recommend modifications with relatively little impact on cost. When changes are requested during software design, the cost impact grows rapidly.

3. Practitioner's/Developer myths

1. More code means better software."

•**Reality:** Quality, maintainability, and simplicity are often more important than quantity. Less code can lead to fewer bugs and easier maintenance.

2. "All bugs can be fixed."

•**Reality:** While many bugs can be resolved, it's impossible to eliminate all of them. The goal is to reduce risk to an acceptable level.

3. "Testing is just a phase."

•**Reality:** Testing should be integrated throughout the development process, not just a final step before release.

4. Documentation is a waste of time."

•**Reality:** Proper documentation is essential for onboarding, collaboration, and future maintenance. It saves time in the long run.

A Generic view of Software Process

Chapter - Topic Covered

- A Layered Technology
- A Process Framework
 - Generic Process Framework Activities
 - Umbrella Activities
- CMMI Level

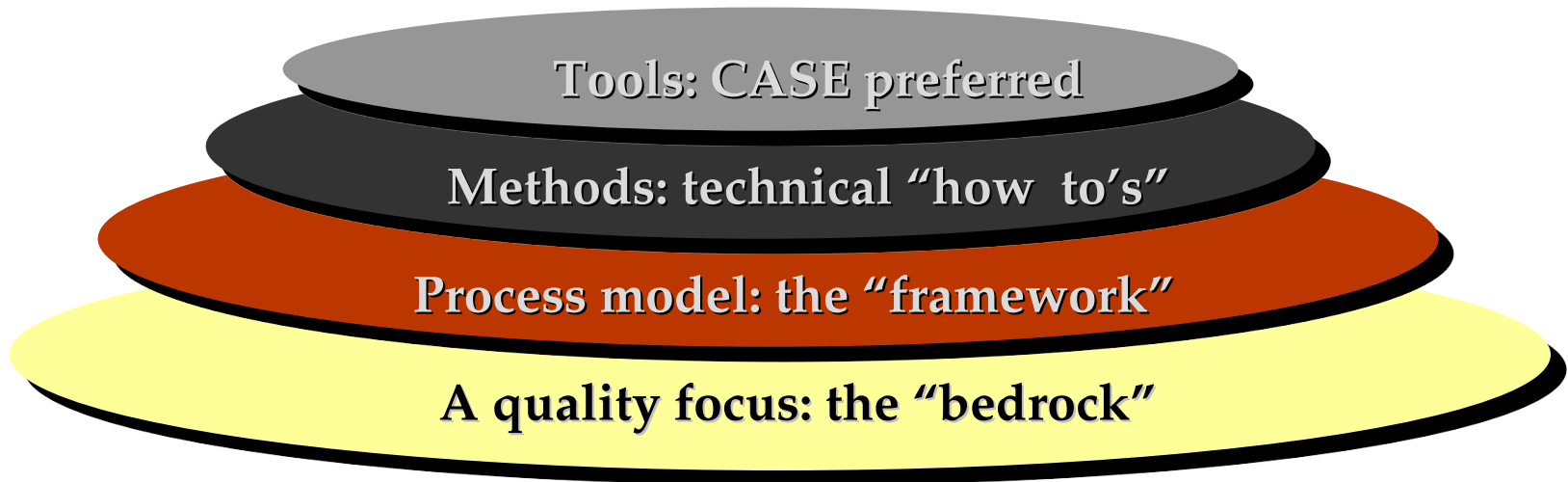
Overview

- **What? A software process** –A series of activities that are required to build high-quality software.
- **Who?** Managers, software engineers, and customers.
- **Steps?** A handful of activities are common to all software processes, and details.
- **Work product?** Programs, documents, and data.
- **A generic view** means general/common/not specific / Non-specialized/Universal / General-purpose.

Software Engineering – Layered Technology

Software engineering is a fully layered technology, to develop software we need to go from one layer to another. All the layers are connected and each layer demands the fulfillment of the previous layer.

Layered Technology



Layered technology is divided into four parts:

1. A quality focus: It defines the continuous process improvement principles of software.

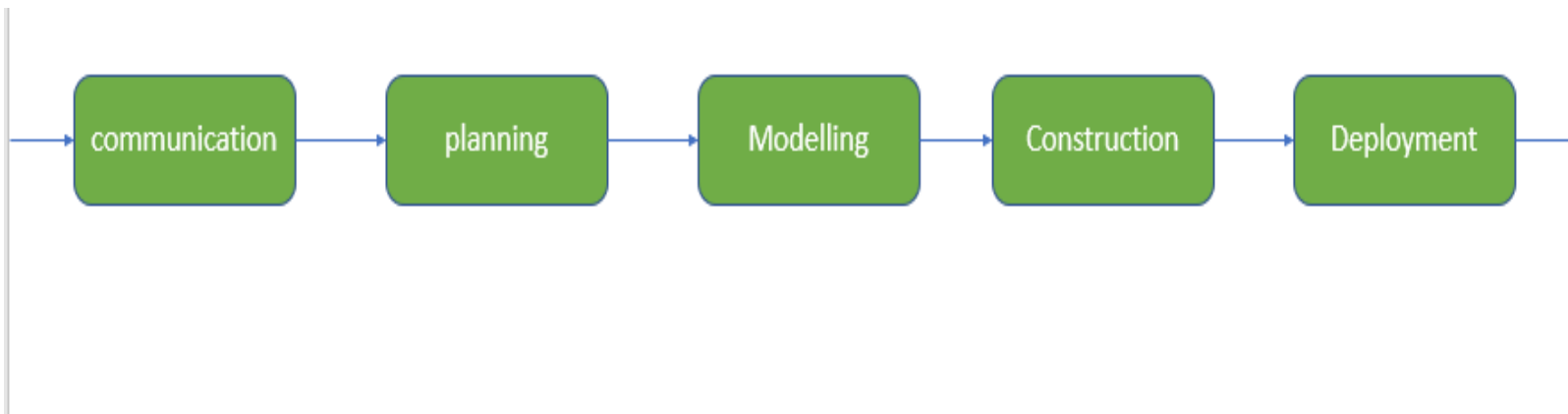
It provides integrity which means providing security to the software so that data can be accessed by only an authorized person, no outsider can access the data. It also focuses on maintainability and usability.

2. Process: It is the foundation or base layer of software engineering.

It is key that binds all the layers together which enables the development of software before the deadline or on time.

Process defines a framework that must be established for the effective delivery of software engineering technology.

The software process covers all the activities, actions, and tasks required to be carried out for software development.



Layered Technology

- **Process activities are listed below:-**

- **Communication:** It is the first and foremost thing for software development.
- Communication is necessary to know the actual demand of the client.
- **Planning:** It means drawing a map to reduce the complications of development.
- **Modeling:** In this process, a model is created according to the client for better understanding.
- **Construction:** It includes the coding and testing of the problem.
- **Deployment:-** It includes the delivery of software to the client for evaluation and feedback

3. Methods:

- Methods encompass a broad array of tasks that include requirements analysis, design, program construction, testing, and support.
- There could be more than one technique to perform a task and different techniques could be used in different situations.

4. Tools:

- Provide automated or semi-automated support for the process, methods and quality control.
- When tools are integrated so that another can use information created by one tool, a system for the support of software development, called ***computer-aided software engineering (CASE)***

Process framework

Provides the foundation for a complete software process.

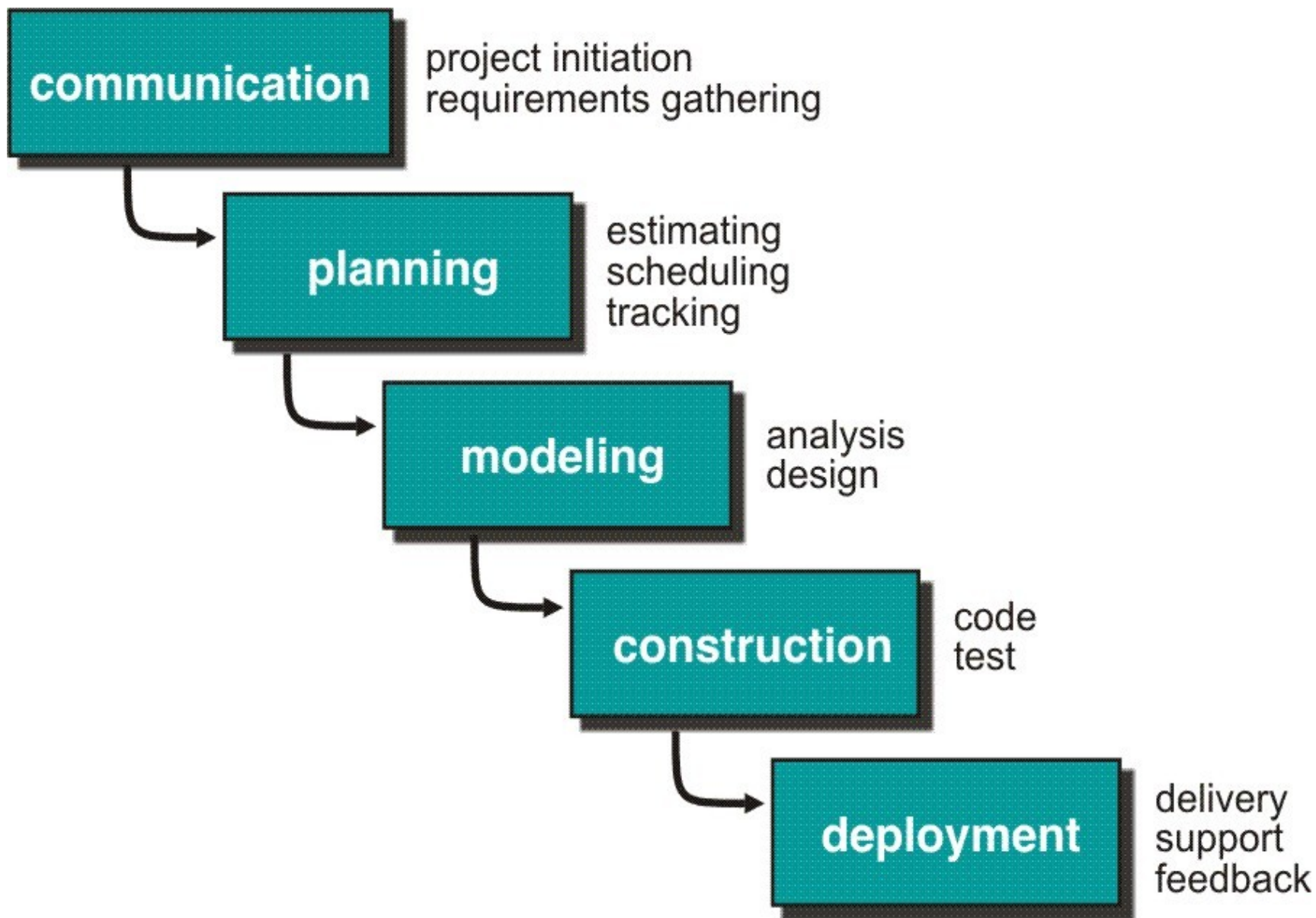
- Identified a small number of **framework activities** that are applicable to all software projects, regardless of their size or complexity.
- It encompasses a set of **umbrella activities** that are applicable across the entire software process.

Process Framework

- Each framework activities is populated by a set *for software engineering actions* – a collection of related tasks.
- Each action has individual *work task*.

Generic Process Framework Activities

- **Communication:**
 - Heavy communication with customers, stakeholders, team
 - Encompasses requirements gathering and related activities
- **Planning:**
 - Workflow that is to follow
 - Describe technical task, likely risk, resources will require, work products to be produced and a work schedule.
- **Modeling:**
 - Help developer and customer to understand requirements (Analysis of requirements) & Design of software
- **Construction**
 - Code generation: either manual or automated or both
 - Testing – to uncover error in the code.
- **Deployment:**
 - Delivery to the customer for evaluation
 - Customer provide feedback



Umbrella Activities

Umbrella activities are a series of steps or procedures followed by a software development team to maintain the progress, quality, changes, and risks of complete development tasks.

- **Software project tracking and control**

- Assessing progress against the project plan.
- Take adequate action to maintain schedule.

- **Formal technical reviews**

- Assessing software work products in an effort to uncover and remove errors before goes into next action or activity.

- **Software quality assurance**

- Define and conducts the activities required to ensure software quality.

- **Software configuration management**

- Manages the effects of change.

- **Document preparation and production**
 - Help to create work products such as models, documents, logs, form and list.
- **Reusability management**
 - Define criteria for work product reuse
 - Mechanisms to achieve reusable components.
- **Measurement**
 - Define and collects process, project, and product measures
 - Assist the team in delivering software that meets customer's needs.
- **Risk management**
 - Assesses risks that may effect that outcome of project or quality of product (i.e. software)

Capability Maturity Model Integration (CMMI)

- The Software Engineering Institute (SEI) has developed a process meta-model to measure an organization's process capability and maturity levels.
- CMMI – developed by SEI
- The CMMI defines each process area in terms of “specific goals” and the “specific practices” required to achieve these goals.
- ***Specific goals*** establish the characteristics that must exist if the activities implied by a process area are to be effective.
- ***Specific practices*** refine a goal into a set of process-related activities.
- A representation allows an organization to pursue a different set of Improvement objectives. There are two representations for CMMI :
- **Staged Representation(maturity Levels)**
- **Continuous Representation(Capability Levels)**

CMMI Level

- **CMMI Model – Maturity Levels**

- In CMMI with staged representation, there are five maturity levels described as follows :

1.Maturity level 1: Initial

1. processes are poorly managed or controlled.
2. unpredictable outcomes of processes involved.
3. No KPAs (Key Process Areas) defined.
4. Lowest quality and highest risk.

2.Maturity level 2: Managed

1. requirements are managed.
2. processes are planned and controlled.
3. projects are managed and implemented according to their documented plans.
4. This risk involved is lower than Initial level, but still exists.
5. Quality is better than Initial level.

3.Maturity level 3 : Defined

1. processes are well characterized and described using standards, proper procedures, and methods, tools, etc.
2. Medium quality and medium risk involved.
- 3.Focus is process standardization.

4.Maturity level 4 : Quantitatively managed

1. quantitative objectives for process performance and quality are set.
2. quantitative objectives are based on customer requirements, organization needs, etc.
3. process performance measures are analyzed quantitatively.
- 4.Higher quality of processes is achieved.
- 5.lower risk

5.Maturity level 5 : Optimizing

1. continuous improvement in processes and their performance.
2. improvement has to be both incremental and innovative.
3. highest quality of processes.
4. lowest risk in processes and their performance.

CMMI models with continuous representation, there are six capability levels as described below

1.Capability level 0 : Incomplete

1. incomplete process – partially or not performed.
2. one or more specific goals of process area are not met.
3. No generic goals are specified for this level.
4. this capability level is same as maturity level 1.

2.Capability level 1 : Performed

2. process performance may not be stable.
3. objectives of quality, cost and schedule may not be met.
4. a capability level 1 process is expected to perform all specific and generic practices for this level.
5. only a start-step for process improvement.

3.Capability level 2 : Managed

3. process is planned, monitored and controlled.
4. managing the process by ensuring that objectives are achieved.
5. objectives are both model and other including cost, quality, schedule.
6. actively managing processing with the help of metrics.

4.Capability level 3: Defined

1. a defined process is managed and meets the organization's set of guidelines and standards.
2. The focus is process standardization.

5.Capability level 4: Quantitatively Managed

1. The process is controlled using statistical and quantitative techniques.
2. Process performance and quality is understood in statistical terms and metrics.
3. quantitative objectives for process quality and performance are established.

6.Capability level 5: Optimizing

1. focuses on continually improving process performance.
2. Performance is improved in both incremental and innovative ways emphasizing on studying the performance results across the organization to ensure that common causes or issues are identified and fixed.